📘 DAY 1: COMPLETE TOPIC-BY-TOPIC MASTER STUDY GUIDE

Detailed Conceptual & Technical Notes for the Lead Trainer

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 1 (Full 6 Hours: 3h Morning Architecture + 3h Afternoon Hands-On Lab)
Curriculum Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

🧭 Master Table of Contents (TOC) — Day 1

- Module 1: Introduction to High Performance Computing (HPC)
  - 1.1 What is High Performance Computing?
  - 1.2 HPC vs Normal Computing (The Architectural Differences)
  - 1.3 Applications of HPC in the Real World
  - 1.4 Parallel Computing Fundamentals (Task vs Data Parallelism)
  - 1.5 The End-to-End HPC Workflow (Problem to Production)
- Module 2: HPC Cluster Architecture & Physical Topology
  - 2.1 HPC Cluster Fundamentals (Why Multiple Connected Machines?)
  - 2.2 Compute Nodes (The Parallel Workhorses)
  - 2.3 Head / Login Node (The Secure Gateway)
  - 2.4 Shared Parallel Storage (Lustre, DAOS, NFS)
  - 2.5 High-Speed Interconnects (InfiniBand, RoCE, Slingshot)
  - 2.6 Single-Node vs Multi-Node Execution Models
- Module 3: Intel HPC Hardware & Silicon Architecture
  - 3.1 Intel Xeon Scalable CPUs & Microarchitecture (Sapphire Rapids)
  - 3.2 Intel Data Center GPUs (Max Series / Ponte Vecchio)
  - 3.3 AI Accelerators — Habana Gaudi 2 & Gaudi 3
  - 3.4 CPU vs GPU vs AI Accelerators (Comparative Analysis)
  - 3.5 Hardware-Aware Computing (Matching Workload to Silicon)
- Module 4: Linux Fundamentals & File System Internals
  - 4.1 Why Linux Dominates 100% of the TOP500 Supercomputers
  - 4.2 Linux File System Hierarchy & Inode Concepts
  - 4.3 Navigating the Cluster: Absolute vs Relative Paths
  - 4.4 File & Directory Management (`mkdir`, `touch`, `cp`, `mv`, `rm`)
  - 4.5 Reading & Inspecting Data Streams (`cat`, `less`, `head`, `tail`)
  - 4.6 High-Speed Search on File Systems (`grep` and `find`)

    - 4.7 Linux Permission Security: Octal Notation & `chmod`
    - 4.8 Environment Inspection (`whoami` , `hostname` , `pwd` , `uname`)
    - 4.9 Environment Variables & `$PATH` Configuration
    - 4.10 Probing Tool Availability (`command -v` , `which`)
- Module 5: Shell Scripting for HPC Automation
    - 5.1 Interactive Shell vs Batch Shell Scripts
    - 5.2 Bash Shell Internals & The Shebang Line (`#!/usr/bin/env bash`)
    - 5.3 Variables, Scope & Command Substitution (`$(...)`)
    - 5.4 Standard I/O Redirection & Streams (`stdin` , `stdout` , `stderr`)
    - 5.5 Defensive Shell Programming (`set -euo pipefail`)
    - 5.6 Automated Node Audit Script (`hpc_node_audit.sh`) Walkthrough
- Module 6: Secured Shell (SSH) & Remote Supercomputing Access
    - 6.1 The Architecture of SSH Protocol (Port 22 & Encryption)
    - 6.2 Asymmetric Cryptography: Public vs Private Key Pairs
    - 6.3 Generating Ed25519 Keys (`ssh-keygen`) & Key Permissions
    - 6.4 Remote Server Authentication (`authorized_keys` & `known_hosts`)
    - 6.5 Remote Command Execution & File Transfer (`scp`)
    - 6.6 Live Demonstration: Connecting to Enterprise Intel Cloud (`ssh idc-vm`)
- Module 7: Intel oneAPI Toolkit (Unified Software Stack)
    - 7.1 The Heterogeneous Dilemma & Intel's Unified Vision
    - 7.2 Khronos SYCL Standard & The Single-Source Model
    - 7.3 Environment Initialization (`source /opt/intel/oneapi/setvars.sh`)
    - 7.4 Device Introspection with `sycl-ls`
    - 7.5 Intel DPC++/C++ Compiler (`icpx -fsycl`)
    - 7.6 Real SYCL Vector Addition Walkthrough & Silicon Validation
- Module 8: Introduction to Intel HPC Toolkit
    - 8.1 Historical Base vs HPC Toolkit & The 2026 Unified Suite
    - 8.2 Intel MPI Library: Multi-Process Distributed Memory Model
    - 8.3 Compiling & Running MPI Applications (`mpicxx` & `mpirun`)
    - 8.4 Intel Fortran Compiler (`ifx`) in Modern Supercomputing
    - 8.5 Performance Profiling Overview (Intel VTune Profiler)

MODULE 1: Introduction to High Performance Computing (HPC)

1.1 What is High Performance Computing?

(Already covered in depth in the introductory lecture notes: Problem deconstruction, why normal laptops fail, compute-bandwidth memory wall, and basic multiprocessing concepts).

1.2 HPC vs Normal Computing (The Architectural Differences)

What is it?

A common computer (laptop/desktop) is designed for **single-user, interactive, latency-sensitive desktop tasks** (web browsing, typing, light compilation). An HPC system is engineered for **massive computational throughput, multi-tenant batch jobs, and extreme memory bandwidth**.

Deep Technical Mechanism:

1. **Memory Channels & Bandwidth Wall:**
   - A standard consumer Intel Core i5/i7/i9 desktop has a dual-channel DDR4/DDR5 memory controller (~50 to 80 GB/s peak bandwidth).
   - An enterprise Dual-Socket Intel Xeon Scalable server (Sapphire Rapids) features **16 memory channels (8 per socket) of DDR5-4800**, providing over **600 GB/s of sustained bandwidth**.
   - If an algorithm performs matrix multiplication or large CFD mesh processing, a desktop CPU spends >85% of its clock cycles waiting for data to arrive from RAM (Cache Misses & Memory Stalls). The Xeon processor keeps its execution ALUs fed.
2. **ECC (Error-Correcting Code) Memory:**
   - Normal laptops use non-ECC RAM. Cosmic rays or electrical fluctuations can flip a single bit in RAM once every few weeks. On a laptop, a flipped bit causes a minor crash or browser glitch.
   - On a supercomputer running a 3-week quantum chemistry simulation across 50 Terabytes of RAM, a single bit flip would silently corrupt scientific results. HPC hardware strictly mandates **ECC DDR5 memory** which detects and corrects single-bit errors on the fly.
3. **Cooling & Thermal Envelope (TDP):**
   - Laptops are thermally constrained to 15W - 45W. After 30 seconds of full load, the CPU throttles clock frequency down by 40% to prevent melting (Thermal Throttling).
   - HPC servers are housed in 1U/2U rack chassis with high-CFM industrial fans or direct-to-chip liquid cooling running continuous 350W TDP per socket without downclocking.

Classroom Example & Analogy:

> *"Students, think of a personal laptop as a fast sports car (Ferrari). It can accelerate to 100 km/h in 3 seconds, carrying 2 passengers. But what if you need to transport 1,000 tons of steel from Mathura to Delhi? You cannot load 1,000 tons into a Ferrari. You need a dedicated freight train with 50 interconnected wagons (an HPC cluster). The freight train may not accelerate as fast as a sports car, but its massive carrying capacity (throughput) completes the work impossible for a car!"*

Common Misconception:

- *Misconception:* "If I buy a gaming laptop with an RTX 4090, I have an HPC supercomputer."
- *Truth:* No! A gaming laptop lacks multi-socket Xeon scalability, has narrow memory channels, lacks ECC memory, and has no high-speed InfiniBand interconnect to communicate with other machines.

Questions to Ask Students:

1. "*If you run an AI training job on your laptop and it hits 100% CPU, why does the laptop fan start screaming and the system slows down?*" (Expected answer: Thermal throttling and memory bus saturation).
2. "*Why do scientific research labs spend millions on Xeon servers instead of buying 50 gaming desktops?*" (Expected answer: Memory bandwidth, ECC reliability, 24/7 sustained thermal stability, and cluster networking).

1.3 Applications of HPC in the Real World

What is it?

HPC is the fundamental engine driving modern science, defense, healthcare, and artificial intelligence.

Key Domains:

1. **Weather Forecasting & Climate Modeling (Numerical Weather Prediction - NWP):**
   - The Earth's atmosphere is divided into a 3D grid of millions of cells. At each grid point, partial differential equations (Navier-Stokes fluid dynamics, thermodynamics, solar radiation) are solved for every 10-second timestep.
   - Without HPC clusters, a 24-hour weather forecast would take 30 days to compute—making the forecast completely useless!
2. **Generative AI & Large Language Models (LLMs):**
   - Training a 70-Billion parameter Transformer model (like Llama-3) requires approximately 10^{24} floating-point operations. Running this sequentially on a single high-end CPU would take over **1,200 years**! An HPC cluster with thousands of accelerators completes it in a few weeks.
3. **Aerodynamic & Automotive Engineering (CFD - Computational Fluid Dynamics):**
   - Companies like Boeing, Formula 1, and Airbus simulate airflow over airplane wings and racing cars inside a virtual wind tunnel using HPC before building physical prototypes, saving billions in manufacturing.
4. **Drug Discovery & Molecular Dynamics:**
   - Simulating how a small drug molecule binds to a COVID-19 or cancer virus protein receptor involves calculating Coulomb and van der Waals forces between 500,000 individual atoms.

1.4 Parallel Computing Fundamentals (Task vs Data Parallelism)

Deep Technical Mechanism:

Sequential computing follows the traditional Von Neumann architecture where an instruction pointer executes one instruction after another:

$$T_{\text{serial}} = N \times t$$

Parallel computing breaks the problem into independent chunks executed concurrently on P processing units.

```
[ SERIAL EXECUTION ]
Task 1 (2s) ---> Task 2 (2s) ---> Task 3 (2s) ---> Task 4 (2s)  == Total Time: 8s

[ PARALLEL EXECUTION (P = 4 Cores) ]
Core 0: Task 1 (2s) --+
Core 1: Task 2 (2s) --+---> Synchronize / Merge Results         == Total Time: ~2.1s
Core 2: Task 3 (2s) --+
Core 3: Task 4 (2s) --+
```

The Two Pillars of Parallelism:

1. **Data Parallelism (SIMD / Array Processing):**
   - The *same operation* is applied to different subsets of data simultaneously.
   - *Example:* Adding two arrays of 1,000,000 numbers: $C[i] = A[i] + B[i]$. Core 0 calculates indices \$0\$ to $250,000$; Core 1 calculates $250,001$ to $500,000$, and so forth.
2. **Task Parallelism (MIMD / Functional Decomposition):**
   - Completely *different functions/tasks* execute concurrently on the same or different data.

- *Example:* In a video game engine, Core 0 calculates physical collisions, Core 1 calculates sound audio rendering, Core 2 executes enemy AI logic, and Core 3 streams assets from disk.

Amdahl's Law (Why 100 Cores Don't Give 100x Speedup):

Gene Amdahl formulated that the maximum theoretical speedup of a program is limited by the **strictly serial fraction (s)** of the code:

$$\text{Speedup}(P) = \frac{1}{s + \frac{1-s}{P}}$$

Takeaway: If only 5% of your code is serial ($s = 0.05$) and 95% is parallel, even if you buy **1,000,000 CPU cores ($P \to \infty$)**, your maximum possible speedup can **NEVER exceed 20x**!

1.5 The End-to-End HPC Workflow (Problem to Production)

```
+-----------------------------------------------------------------------------------------------+
|                                  THE COMPLETE HPC WORKFLOW LIFECYCLE                          |
+-----------------------------------------------------------------------------------------------+
| 1. PROBLEM FORMULATION    : Identify computational bottlenecks (e.g. O(N^3) matrix math, CFD mesh) |
|            |                                                                                  |
|            v                                                                                  |
| 2. LOCAL PROTOTYPING      : Write serial C++/Python code, validate math on small test dataset      |
|            |                                                                                  |
|            v                                                                                  |
| 3. CODE PARALLELIZATION   : Introduce oneAPI SYCL, OpenMP (Shared Mem) or MPI (Distributed Mem)    |
|            |                                                                                  |
|            v                                                                                  |
| 4. REMOTE CLUSTER ACCESS  : Connect via SSH (Port 22) to Login Node with Ed25519 Keys             |
|            |                                                                                  |
|            v                                                                                  |
| 5. COMPILATION & BUILD    : Compile on Login node using Intel icpx/ifx with -O3 optimization flags |
|            |                                                                                  |
|            v                                                                                  |
| 6. JOB SCHEDULING (SLURM) : Write batch script, request N compute nodes, submit via sbatch         |
|            |                                                                                  |
|            v                                                                                  |
| 7. EXECUTION & STORAGE    : Nodes compute in parallel, reading/writing to /scratch Lustre NVMe     |
|            |                                                                                  |
|            v                                                                                  |
| 8. PROFILING & TUNING     : Inspect performance bottlenecks using Intel VTune Profiler            |
+-----------------------------------------------------------------------------------------------+
```

MODULE 2: HPC Cluster Architecture & Physical Topology

2.1 HPC Cluster Fundamentals (Why Multiple Connected Machines?)

Moore's Law (transistor scaling) and Dennard Scaling (frequency scaling) broke down around 2005 due to thermal limits (the "Power Wall"). CPU manufacturers could no longer increase clock speeds from 4GHz to 10GHz without melting the silicon. The only path forward to achieve higher compute capacity is **horizontal scaling (Clustering)**.

2.2 Compute Nodes vs 2.3 Head/Login Node



The Golden Operational Rules:

- **Rule 1 (The Cardinal Sin):** Never execute long-running compute jobs (`./simulation`) on the Login Node! The login node has shared memory; if one student consumes 100% CPU on the login node, it freezes the terminal for all 60 students and the trainer.
- **Rule 2:** The Login node is strictly for editing scripts (`nano`), compiling code (`icpx`), and launching batch jobs (`sbatch`). The actual heavy lifting is automatically dispatched by the scheduler to the backend Compute Nodes.

2.4 Shared Parallel Storage (Lustre / DAOS / NFS)

In a cluster of 100 nodes, each node cannot have its own completely isolated copy of a 10-Terabyte dataset.

- **NFS (Network File System):** Good for user home directories (`/home/username`). Simple, but centralized. If 100 nodes try to read a 10GB file from NFS simultaneously, the central NFS server crashes.
- **Parallel File Systems (Lustre / DAOS):** Files are cut into stripes (e.g., 1MB chunks) and distributed across dozens of Object Storage Targets (OSTs). When 100 compute nodes read the file, each node reads from a different storage controller concurrently, achieving aggregate speeds over **1,000 Gigabytes per second**!

2.5 High-Speed Interconnects (InfiniBand vs Standard Ethernet)

- Standard Gigabit Ethernet has a round-trip latency of ~50 to 100 microseconds.
- **InfiniBand (HDR 200 Gbps / NDR 400 Gbps):** Has an ultra-low latency of **sub-1 microsecond (0.6 μs)**.

- **RDMA (Remote Direct Memory Access):** Allows Node A to directly write bytes into the RAM of Node B across the network cable without interrupting the operating system kernel or CPU on Node B!

MODULE 3: Intel HPC Hardware & Silicon Architecture

3.1 Intel Xeon Scalable CPUs (Sapphire Rapids / Emerald Rapids)

- **Die Topology:** Multi-tile modular architecture built on Intel 7 process.
- **On-Die Interconnect:** Intel Ultra Path Interconnect (UPI 2.0) running at 16 GT/s for multi-socket coherency.
- **Vector Engine (AVX-512):** Features dual 512-bit FMA (Fused Multiply-Add) execution pipelines per core. In one clock cycle, a single core can execute:

$$2 \times 16 = 32 \text{ Single-Precision (FP32) Flops per cycle}$$

- **Intel AMX (Advanced Matrix Extensions):** Two dimensional systolic matrix multiply hardware (`TMUL`) supporting Bfloat16 and INT8 formats. Capable of executing **2048 INT8 operations per core per cycle**!

3.2 Intel Data Center GPU Max Series (Ponte Vecchio)

- Designed specifically for HPC and scientific simulations.
- Contains up to 128 Xe-Cores, 128 Ray Tracing Units, and up to 128GB of high-speed HBM2e memory delivering over 3.2 TB/s bandwidth.
- Native hardware support for FP64 (Double Precision) calculations, making it ideal for scientific simulations that require extreme decimal accuracy.

3.3 AI Accelerators — Habana Gaudi 2 & Gaudi 3

- Specifically designed for training and inferencing large language models.
- **Heterogeneous Architecture:** Instead of traditional GPU streaming multiprocessors, Gaudi features custom **Tensor Processor Cores (TPC)** paired with dedicated GEMM (General Matrix Multiply) engines.
- **Integrated Interconnect:** Integrates 24 standard 100-Gigabit Ethernet (RoCEv2) ports directly onto the ASIC. This allows building massive 1,000+ accelerator AI clusters using standard, cost-effective Ethernet switches without requiring expensive proprietary InfiniBand switches!

3.4 Silicon Comparison Summary Table:

Silicon Engine	Primary Architectural Role	Typical Precision Strengths	Key Hardware Innovation
Intel Xeon Scalable CPU	System orchestration, OS, serial logic, branchy code, high single-thread speed.	FP64, FP32, INT8/BF16 (via AMX)	Built-in Intel AMX matrix tiles, massive shared L3 Cache, 8-channel DDR5.
Intel Data Center GPU Max	Massively parallel scientific computations, molecular dynamics, CFD, graphics.	FP64, FP32, FP16	Xe-Cores, Foveros 3D packaging, 128GB HBM2e (3.2 TB/s bandwidth).
Habana Gaudi 2 / 3	Deep Learning tensor math, Large Language Model (LLM) training and inference.	BF16, FP8, INT8	On-chip 24x 100GbE RoCE networking, specialized Tensor Processor Cores (TPC).

MODULE 4: Linux Fundamentals & File System Internals

4.1 Why Linux Dominates 100% of the TOP500 Supercomputers

Since November 2017, all 500 of the world's fastest supercomputers run on Linux.

1. **Open Source & Kernel Customizability:** HPC engineers can strip down unnecessary background GUI services, tune the CFS (Completely Fair Scheduler), and configure huge memory pages (`HugePages 2MB/1GB`).
2. **Modular Headless Operation:** Zero overhead from display managers or graphical desktops; 100% of CPU and RAM is dedicated to mathematical computation.
3. **Rock-Solid Stability:** Linux servers regularly achieve years of continuous uptime without rebooting.

4.2 Linux File System Hierarchy & Inode Concepts

In Linux, *everything is a file* (including storage devices, network sockets, and processes in `/proc`).

- `/` **(Root):** Apex of the directory tree.
- **Inode (Index Node):** A data structure on Linux filesystems that stores all metadata about a file (file size, physical disk block locations, owner user ID, permissions, timestamps), except for its filename and actual data!

4.6 High-Speed Search on File Systems (`grep` vs `find`)

- `grep` **:** Searches *inside* files for matching textual patterns.

```
# Search for the string "error" case-insensitively inside all .log files
grep -i "error" *.log
```

- `find` **:** Searches *the filesystem directory tree* for matching filenames, sizes, or modification dates.

```
# Find all C++ source files modified in the last 24 hours
find ~/hpc_day01 -name "*.cpp" -mtime -1
```

4.7 Linux Permission Security: Octal Notation & `chmod`

Permissions are represented by 3 digits (Owner, Group, Others):

$$\text{Read } (r) = 4, \quad \text{Write } (w) = 2, \quad \text{Execute } (x) = 1$$

Octal Value	Binary	Symbolic Representation	Typical Usage
`755`	`111 101 101`	`rwxr-xr-x`	Executable scripts and public directories.
`644`	`110 100 100`	`rw-r--r--`	Standard text and data files (readable by all, modifiable only by owner).
`600`	`110 000 000`	`rw-------`	Private cryptographic keys (SSH keys, passwords). Strictly enforced!

MODULE 5: Shell Scripting for HPC Automation

5.2 The Shebang Line (`#!/usr/bin/env bash`)

When you execute `./script.sh` , the Linux kernel does not assume the script is written in Bash. It inspects the first two bytes: `0x23 0x21` (`#!`).

- `#!/bin/bash`: Hardcoded path. Fails if Bash is installed in `/usr/local/bin` or custom container path.
- `#!/usr/bin/env bash`: **Best Practice.** Queries the active `$PATH` environment to find the current user's default Bash interpreter.

5.5 Defensive Shell Programming (`set -euo pipefail`)

By default, Bash continues running even if a command fails!

```
#!/bin/bash
TARGET_DIR="/path/does/not/exist"
cd $TARGET_DIR
rm -rf *  # DISASTER! Since cd failed, this deletes everything in the current working directory!
```

Adding `set -euo pipefail` stops execution instantly if any directory or variable fails, preventing data loss!

MODULE 6: Secured Shell (SSH) & Remote Access

6.2 Asymmetric Cryptography: Public vs Private Key Pairs

- Traditional passwords transmit a secret string over the network. Even with encryption, passwords can be brute-forced or intercepted by malicious keyloggers.
- **Asymmetric Key Exchange (Ed25519 / Curve25519):**
  1. The client generates a mathematical key pair: Private Key (`id_ed25519`) and Public Key (`id_ed25519.pub`).
  2. The Public Key is placed in the remote server's `~/.ssh/authorized_keys`.
  3. When connecting, the server creates a random cryptographic challenge, encrypts it using the Public Key, and sends it to the client.
  4. The client decrypts the challenge using its local Private Key and returns the proof.
  5. **The Private Key never travels across the network cable!**

MODULE 7: Intel oneAPI Toolkit (Unified Software Stack)

7.1 The Heterogeneous Dilemma & Intel's Unified Vision

Before oneAPI, cross-architecture programming was a nightmare:

- NVIDIA GPUs required proprietary **CUDA**.
- CPUs required **C++ / OpenMP**.
- DSPs and mobile accelerators required custom proprietary SDKs.

Intel oneAPI provides an open, standards-based solution based on Khronos SYCL:

- Write one single modern C++ codebase.
- Compile to Intel Xeon CPUs, Intel Arc GPUs, Intel Data Center GPUs, and (via Codeplay open-source plugins) NVIDIA and AMD GPUs!

7.4 Device Introspection with `sycl-ls`

The `sycl-ls` tool enumerates all hardware compute targets detected by the oneAPI runtime:

```
[opencl:cpu:0] Intel(R) OpenCL, 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
[ext_oneapi_level_zero:gpu:0] Intel(R) Level-Zero, Intel(R) UHD Graphics
```

MODULE 8: Introduction to Intel HPC Toolkit

8.2 Intel MPI Library: Multi-Process Distributed Memory Model

- **What is MPI?** Message Passing Interface is the global standard for distributed cluster computing.
- **Architecture:** Unlike multithreading (where threads share the same RAM), MPI processes each have their own **completely private, isolated memory space**.
- **Communication:** Processes send and receive explicit messages across the cluster fabric:
  - `MPI_Send` / `MPI_Recv` (Point-to-point)
  - `MPI_Bcast` / `MPI_Reduce` (Collective communications)

8.3 Compiling & Running MPI Applications

```
# 1. Compile C++ code using Intel MPI wrapper
mpicxx -O2 mpi_hello.cpp -o mpi_hello

# 2. Launch 4 parallel processes on the node
mpirun -n 4 ./mpi_hello
```

Output:

```
>> [Intel HPC Cluster: idc-vm] Process Rank 0 of 4 is actively executing!
>> [Intel HPC Cluster: idc-vm] Process Rank 1 of 4 is actively executing!
>> [Intel HPC Cluster: idc-vm] Process Rank 2 of 4 is actively executing!
>> [Intel HPC Cluster: idc-vm] Process Rank 3 of 4 is actively executing!
```

Dossier Status: Complete, Exhaustive & Formatted | GLA University HPC Masterclass EOF